

Parte A - Tarea 1 Base de Datos

Integrantes:

- Josefa Buch Soto
- Matías Salinas Duarte
- Constanza Venegas Vera

1. Se le pidió a la LLM (Gemini - Google IA Pro) que diseñe un esquema a partir de los requerimientos del cliente proporcionados. El **prompt** utilizado fue el siguiente:

Crea un esquema relacional en base a los requerimientos a continuación. Evita lo más posible redundancias e intenta seguir la forma normal de Boyce-Codd.

Aquí están los requerimientos:

Torneos. La organización realiza múltiples torneos. Cada torneo tiene un nombre, el título del video juego que se juega, fechas de inicio y fin, y un pozo de premios (prize pool) en USD. Además tiene un número máximo de equipos por participar.

Jugadores. Los jugadores se registran en la plataforma con un gamertag (único), su nombre real, email, fecha de nacimiento y país de origen.

Equipos. Los jugadores forman equipos (de un número no acotado de jugadores). Cada equipo tiene un nombre único, una fecha de creación y un capitán. El capitán debe ser miembro del equipo. Un jugador pertenece a exactamente un equipo (la membresía es fija, no cambia entre torneos).

Inscripción a torneos. Los equipos se inscriben en torneos. Cada torneo tiene un número máximo de equipos que pueden participar. Un equipo puede inscribirse en múltiples torneos.

Partidas. Dentro de un torneo se juegan partidas (matches). Cada partida enfrenta a exactamente dos equipos distintos, tiene una fecha y hora programada, un resultado (puntaje equipo A y puntaje equipo B) y una fase del torneo. Las fases posibles son: fase de grupos, cuartos de final, semifinal, final. El formato del torneo es: 8 equipos divididos en 2 grupos de 4, round-robin dentro de cada grupo, los 2 mejores de cada grupo avanzan a semifinales, luego final.

Estadísticas individuales. Para cada partida se registran las estadísticas de cada jugador de los equipos participantes: KOs, restarts y assists.

Sponsors. Empresas pueden auspiciar torneos. De cada sponsor se conoce su nombre, industria (tecnología, bebidas, ropa, etc.) y el monto en USD que aporta a cada torneo que auspicia. Un sponsor puede auspiciar múltiples torneos.

La respuesta de la LLM fue la siguiente:

Aquí tienes el diseño del esquema relacional en SQL (optimizado para motores como PostgreSQL) que cumple con todos tus requerimientos.

*Para alcanzar la ****Forma Normal de Boyce-Codd (BCNF)****, el esquema garantiza que todo determinante sea una clave candidata. Se han separado las relaciones muchos a muchos (Sponsors-Torneos, Equipos-Torneos, Jugadores-Partidas) en tablas intermedias para evitar cualquier redundancia de datos.*

1. Equipos y Jugadores

Existe una dependencia circular aquí: un equipo necesita un capitán (que es un jugador), y un jugador debe pertenecer a un equipo. Para resolver esto a nivel de base de datos sin romper las restricciones, creamos la tabla `Equipos` primero sin la llave foránea del capitán, creamos los `Jugadores`, y luego alteramos la tabla `Equipos`.

```
```sql
CREATE TABLE Equipos (
 id_equipo SERIAL PRIMARY KEY,
 nombre VARCHAR(100) UNIQUE NOT NULL,
 fecha_creacion DATE NOT NULL,
 id_capitan INT UNIQUE -- Será llave foránea a Jugadores. UNIQUE asegura que un jugador solo
 capitaneé un equipo a la vez.
);
```

```
CREATE TABLE Jugadores (
 id_jugador SERIAL PRIMARY KEY,
 gamertag VARCHAR(50) UNIQUE NOT NULL,
 nombre_real VARCHAR(150) NOT NULL,
 email VARCHAR(150) UNIQUE NOT NULL,
 fecha_nacimiento DATE NOT NULL,
 pais_origen VARCHAR(50) NOT NULL,
 id_equipo INT NOT NULL, -- NOT NULL garantiza que pertenece a exactamente 1 equipo
 CONSTRAINT fk_jugador_equipo FOREIGN KEY (id_equipo) REFERENCES Equipos(id_equipo)
);
```

*-- Añadimos la relación del capitán una vez que la tabla Jugadores existe*

```
ALTER TABLE Equipos
ADD CONSTRAINT fk_equipo_capitan FOREIGN KEY (id_capitan) REFERENCES
Jugadores(id_jugador);
```
```

(Nota de integridad: Para garantizar estrictamente que `id\_capitan` pertenezca al mismo `id\_equipo`, usualmente se implementa un `TRIGGER` o una validación en la capa de la aplicación (como en Python/Flask) antes de insertar).

2. Torneos e Inscripciones

Separamos la información general del torneo de las inscripciones de los equipos.

```

```sql
CREATE TABLE Torneos (
 id_torneo SERIAL PRIMARY KEY,
 nombre VARCHAR(150) NOT NULL,
 titulo_videojuego VARCHAR(100) NOT NULL,
 fecha_inicio DATE NOT NULL,
 fecha_fin DATE NOT NULL,
 prize_pool_usd DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
 max_equipos INT NOT NULL CHECK (max_equipos > 0),
 CONSTRAINT check_fechas_torneo CHECK (fecha_fin >= fecha_inicio)
);

CREATE TABLE Inscripciones_Torneo (
 id_torneo INT NOT NULL,
 id_equipo INT NOT NULL,
 fecha_inscripcion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 PRIMARY KEY (id_torneo, id_equipo),
 CONSTRAINT fk_inscripcion_torneo FOREIGN KEY (id_torneo) REFERENCES
Torneos(id_torneo),
 CONSTRAINT fk_inscripcion_equipo FOREIGN KEY (id_equipo) REFERENCES
Equipos(id_equipo)
);
```

```

3. Partidas y Estadísticas

La tabla de partidas conecta a dos equipos bajo un torneo específico. Las estadísticas individuales se asocian a la combinación única de `Partida + Jugador`.

```

```sql
CREATE TABLE Partidas (
 id_partida SERIAL PRIMARY KEY,
 id_torneo INT NOT NULL,
 id_equipo_a INT NOT NULL,
 id_equipo_b INT NOT NULL,
 fecha_hora TIMESTAMP NOT NULL,
 puntaje_equipo_a INT DEFAULT 0,
 puntaje_equipo_b INT DEFAULT 0,
 fase VARCHAR(50) NOT NULL CHECK (fase IN ('fase de grupos', 'cuartos de final', 'semifinal',
'final')),
 CONSTRAINT fk_partida_torneo FOREIGN KEY (id_torneo) REFERENCES Torneos(id_torneo),
 CONSTRAINT fk_partida_equipo_a FOREIGN KEY (id_equipo_a) REFERENCES
Equipos(id_equipo),
 CONSTRAINT fk_partida_equipo_b FOREIGN KEY (id_equipo_b) REFERENCES
Equipos(id_equipo),
 CONSTRAINT check_equipos_distintos CHECK (id_equipo_a <> id_equipo_b) -- Un equipo no
puede jugar contra sí mismo
);
```

```

```
CREATE TABLE Estadisticas_Jugador_Partida (
    id_partida INT NOT NULL,
    id_jugador INT NOT NULL,
    kos INT NOT NULL DEFAULT 0 CHECK (kos >= 0),
    restarts INT NOT NULL DEFAULT 0 CHECK (restarts >= 0),
    assists INT NOT NULL DEFAULT 0 CHECK (assists >= 0),
    PRIMARY KEY (id_partida, id_jugador),
    CONSTRAINT fk_stats_partida FOREIGN KEY (id_partida) REFERENCES Partidas(id_partida),
    CONSTRAINT fk_stats_jugador FOREIGN KEY (id_jugador) REFERENCES Jugadores(id_jugador)
);
'''
```

4. Sponsors y Auspicios

Si colocáramos el "monto en USD" dentro de la tabla del sponsor o del torneo, romperíamos la BCNF (ya que el monto depende de *qué sponsor* auspicia *qué torneo*). Por eso va en una tabla puente.

```
```sql
CREATE TABLE Sponsors (
 id_sponsor SERIAL PRIMARY KEY,
 nombre VARCHAR(100) NOT NULL,
 industria VARCHAR(100) NOT NULL
);

CREATE TABLE Auspicios (
 id_sponsor INT NOT NULL,
 id_torneo INT NOT NULL,
 monto_usd DECIMAL(12, 2) NOT NULL CHECK (monto_usd > 0),
 PRIMARY KEY (id_sponsor, id_torneo),
 CONSTRAINT fk_auuspicio_sponsor FOREIGN KEY (id_sponsor) REFERENCES Sponsors(id_sponsor),
 CONSTRAINT fk_auuspicio_torneo FOREIGN KEY (id_torneo) REFERENCES Torneos(id_torneo)
);
'''
```

#### ### Resumen de justificación BCNF:

1. **\*\*No hay dependencias transitivas:\*\*** En `Jugadores`, atributos como `email` o `gamertag` son únicos (claves candidatas), y el resto de los datos dependen completamente de ellos o de la clave primaria (`id\_jugador`).
2. **\*\*Aislamiento de la lógica de negocio:\*\*** El formato del torneo (grupos de 4, avance a semifinales) es una lógica de la aplicación, por lo que la base de datos simplemente proporciona la infraestructura en la columna `fase` de la tabla `Partidas` para que el backend lo organice.
3. **\*\*Evitación de anomalías de actualización:\*\*** Si un Sponsor cambia de industria, se actualiza solo en un registro en la tabla `Sponsors`, sin afectar el histórico de sus `Auspicios`.

2. Luego, se unificaron los esquemas individuales del grupo con el fin de comparar el resultado final con el de la LLM. El esquema resultante en esta primera instancia fue el siguiente:

**Entidades:**

- Torneos(nombre str(30) PRIMARY KEY,  
videojuego str(30),  
fecha\_inicio date PRIMARY KEY,  
fecha\_termino date,  
pozo float,  
equipos\_incritos int)
- Jugadores(gamertag str(30) PRIMARY KEY,  
nombre str(30),  
email str(30),  
fecha\_nacimiento date,  
pais\_origen str(30))
- Equipos(nombre char(30) PRIMARY KEY,  
fecha\_creacion date,  
capitán str(30)) FOREIGN KEY
- Sponsors(nombre char(30) PRIMARY KEY,  
industria char(30),  
monto int)

**Relaciones:**

- Es\_del\_equipo(gamertag char(30) FOREIGN/PRIMARY\_KEY,  
nombre\_equipo char(30) FOREIGN/PRIMARY\_KEY)
- Esta\_en\_torneo(nombre\_equipo char(30) PRIMARY KEY,  
nombre\_torneo char(30) PRIMARY KEY,  
fecha\_inicio date PRIMARY KEY)
- Partida(  
ID\_partida int PRIMARY\_KEY  
nombre\_equipo1 char(30) FOREIGN KEY,  
nombre\_equipo2 char(30) FOREIGN KEY,  
inicio timestamp,  
puntaje\_equipo1 int,  
puntaje\_equipo2 int,  
fase char(30))
- Estadisticas\_en\_partida(gamertag char(30) PRIMARY KEY,  
ID\_partida int PRIMARY KEY  
KOs int,  
restarts int,  
assists int)

```
- Sponsor_del_torneo(nombre_sponsor char(30) PRIMARY KEY,
 nombre_torneo char(30) PRIMARY KEY,
 fecha_torneo date PRIMARY KEY)
```

A partir de ambos esquemas, es posible realizar un análisis crítico:

#### **Fortalezas y aspectos positivos de la LLM:**

1. **Atributo sobre equipos en entidad Torneos:** La LLM sustituyó el atributo *equipos\_inscritos* por *max\_equipos*. Esto debido a que la cantidad de equipos inscritos es un valor calculado o derivado de consultas a la base de datos, por lo que se debe eliminar. Fue reemplazado por la cantidad de equipos máximo que puede contener, ya que es uno de los requisitos del cliente: cada torneo “tiene un número máximo de equipos por participar”.
2. **Referencia a Torneos desde entidad Partidas:** en la tabla Partidas se incluyó la llave primaria compuesta de Torneos (*nombre\_torneo* + *fecha\_inicio*). De esta forma se permite vincular la partida con su torneo de origen.
3. **Monto de auspicio:** el atributo *monto* que entrega cada auspiciador varía según el torneo. Por lo tanto, si se coloca dentro de la entidad Sponsors, este monto sería fijo por cada auspiciador. De esta forma, es correcto que se encuentre dentro de la relación Sponsor\_del\_Torneo para que indique el monto de auspicio para cada torneo.

#### **Correcciones a la LLM:**

1. **Redundancia en llaves primarias:** se eliminaron los atributos del tipo *id* numéricos en las tablas Equipos y Jugadores. Esto se debe a que el *nombre* (en Equipos) y el *gamertag* (en Jugadores) son únicos y son utilizados como primary keys, siendo redundante el uso de nuevos id's.
2. **Dependencia Circular:** dentro de la tabla Jugadores, la LLM agregó el atributo *id\_equipo*. El problema de esto recae en la dependencia circular producida con la tabla Equipos. Lo anterior ocurre ya que, de manera simultánea, se requiere de un *id\_capitán* en Equipos y un *id\_equipo* en Jugadores. De esta forma, cada tabla necesita información de la otra para ser creada. Como solución, el atributo *id\_equipo* en Jugadores no fue considerado en el esquema, y en su reemplazo se mantiene la relación Es\_del\_Equipo. Con esto, el *gamertag* actúa como primary key de la relación, asegurando que cada jugador pertenezca a un único equipo y que la dependencia circular sea eliminada.